Final Project Report
Adversarial Robustness Analysis of Neural Network-Based Intrusion Detection Systems Using FGSM and PGD Attacks

Conrad Miller Prathik Bengaluru Prabhakara Bhargav Yellepeddi

May 2026

1 Project Overview

This project studies adversarial robustness in a cybersecurity intrusion detection setting. We train a Multi-Layer Perceptron (MLP) classifier on the NSL-KDD dataset to distinguish normal traffic from attack traffic, evaluate its behavior under FGSM and PGD white-box attacks across multiple perturbation strengths, and then compare it against an adversarially trained version of the same model. The goal is to quantify how vulnerable a standard neural network IDS is to adversarial evasion and to measure whether adversarial training provides a meaningful defense.

2 Motivation and Relevance

Machine learning models are increasingly used in intrusion detection systems because they can achieve very high accuracy on labeled traffic data. However, these models can also be fragile: small, carefully constructed perturbations to the input can cause a model to misclassify malicious traffic as benign. In a security setting, this is especially important because the attacker does not need to fully change the semantic behavior of the traffic. Instead, the attacker only needs to modify feature values enough to evade detection.

This project is relevant because it evaluates that risk in a controlled IDS setting and measures whether a widely used defense, adversarial training, can improve robustness without significantly harming clean-data accuracy. The project also highlights the difference between nominal model performance and adversarial robustness, which is a critical distinction in security-sensitive machine learning deployments.

3 Methodology and Implementation

3.1 Dataset Preparation

We used the NSL-KDD dataset, a benchmark dataset for intrusion detection research containing labeled network connection records. The original class labels were converted into binary targets: normal traffic was mapped to 1, while all attack classes were mapped to 0. This formulation matches the core objective of normal-versus-attack classification under adversarial perturbations. The final dataset contained 151,165 records. After conversion into binary labels, it remained reasonably balanced, with approximately 53.45% normal traffic and 46.55% attack traffic. This helped confirm that clean-data performance was not being driven by a severe class-imbalance artifact.

3.2 Preprocessing

The dataset contains both continuous features, such as duration and byte counts, and categorical features, such as protocol type and service. Categorical features were one-hot encoded and continuous features were standardized. During adversarial evaluation, perturbations were applied only to continuous features, while one-hot categorical columns were preserved so that perturbed samples remained semantically valid. After adversarial perturbation, continuous features were clipped to observed training-set bounds to keep generated samples within realistic ranges. After preprocessing, each network connection was represented as a 120-dimensional numeric feature vector combining scaled continuous features and one-hot encoded categorical features.

3.3 Data Splitting

We used a stratified train-validation-test split to preserve class balance across all subsets. The final split was approximately 70% training, 15% validation, and 15% testing. The validation set supported model monitoring during training, while the test set was reserved for final clean and adversarial evaluation.

3.4 Model Architecture and Training

The base model was an MLP with an input layer matching the processed feature dimension, a first hidden layer with 64 neurons and ReLU activation, a second hidden layer with 32 neurons and ReLU activation, and a single-logit output layer for binary classification. We used `BCEWithLogitsLoss` as the loss function and Adam with a learning rate of 0.001. The baseline model was trained for 10 epochs with a batch size of 64.

3.5 FGSM Attack Implementation

We implemented the Fast Gradient Sign Method (FGSM) as a single-step white-box attack under an L_∞ constraint. FGSM computes the gradient of the loss with respect to the input, takes the sign of that gradient, and perturbs each continuous feature by ε in the direction that maximizes the loss. We evaluated FGSM at $\varepsilon \in \{0.0, 0.01, 0.05, 0.10, 0.20, 0.30\}$.

3.6 PGD Attack Implementation

We also implemented Projected Gradient Descent (PGD), an iterative variant of FGSM. PGD begins from a random point inside the ε-ball and then performs repeated gradient-sign updates, projecting the perturbation back into the valid L_∞ ball after each step. In our experiments, PGD used 10 steps with a step size of $\varepsilon/4$. As with FGSM, perturbations were restricted to continuous features and clipped to valid data ranges.

3.7 Adversarial Training

To improve robustness, we trained a second MLP using adversarial training. During each training batch, FGSM adversarial examples were generated on the fly using the current model weights with $\varepsilon = 0.1$. Clean samples and adversarial samples were then combined into a single training batch, effectively doubling the batch size from 64 to 128. This process encouraged the model to learn decision boundaries that were less sensitive to small adversarial perturbations.

4 Adversarial ML Model Architecture

Figure 1 illustrates the end-to-end pipeline used in this project, from dataset ingestion through final evaluation. The architecture consists of five major stages: data preparation, model training, adversarial

example generation, validity enforcement, and comparative evaluation.

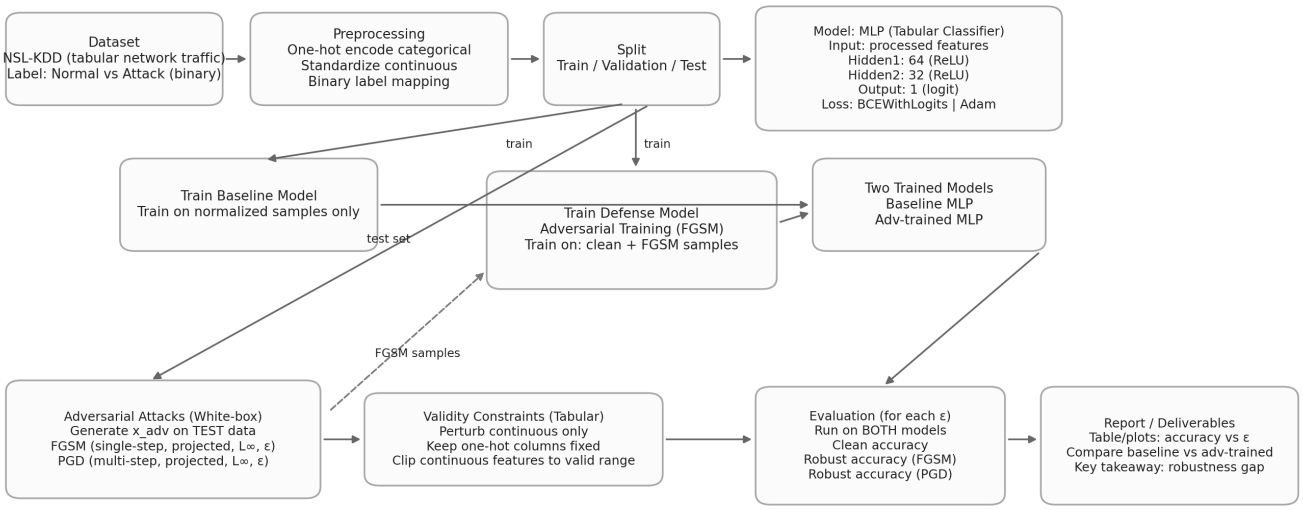

Figure 1: End-to-end adversarial ML pipeline for intrusion detection. Solid arrows indicate the primary data flow; dashed arrows indicate adversarial sample generation during training and evaluation.

The pipeline begins with the NSL-KDD dataset, which is preprocessed by one-hot encoding categorical features, standardizing continuous features, and mapping the original labels to a binary normal-versus-attack target. The processed data is then split into stratified train, validation, and test sets.

From the training data, two models are produced. The baseline MLP is trained on clean normalized samples only. The adversarially trained MLP is trained on a combined batch of clean samples and FGSM-generated adversarial samples created on the fly at $\varepsilon = 0.1$. Both models share the same architecture: an input layer matching the number of processed features, a 64-neuron hidden layer with ReLU activation, a 32-neuron hidden layer with ReLU activation, and a single-logit output layer optimized with `BCEWithLogitsLoss` and Adam.

At evaluation time, adversarial examples are generated from the held-out test set using both FGSM and PGD under L_∞ constraints across multiple ε values. A validity constraint module ensures that perturbations are applied only to continuous features, one-hot categorical encodings remain intact, and all perturbed continuous values are clipped to their observed training-set bounds. Both the baseline and adversarially trained models are then evaluated on these constrained adversarial inputs to produce the comparative robustness results reported in Table 2.

5 Results and Discussion

5.1 Baseline Evaluation on Clean Data

After training, the baseline model achieved the following performance on the clean, unperturbed test set.

The adversarially trained model achieved a clean accuracy of 99.56%, showing that robustness improvements did not come with a meaningful loss in standard predictive performance. The high clean-data performance is also expected for this dataset because NSL-KDD contains strong, learnable feature patterns. In particular, features such as service, flag, byte count, and error-rate features provide strong indicators of attack versus normal behavior. That makes the dataset a useful baseline for robustness testing because the later drops in accuracy can be attributed to adversarial perturbations rather than to an already weak classifier.

Metric	Value
Accuracy	99.53%
Precision	99.59%
Recall	99.54%
F1-score	99.56%
ROC-AUC	0.9998
PR-AUC	0.9998

Table 1: Baseline performance on clean test data.

5.2 Baseline vs. Adversarially Trained Model

Table 2 compares both models under FGSM and PGD across the full epsilon sweep.

ε	Base FGSM	Adv FGSM	Base PGD	Adv PGD
0.00	99.53%	99.56%	99.53%	99.56%
0.01	99.43%	99.55%	99.43%	99.55%
0.05	98.83%	99.48%	98.78%	99.47%
0.10	96.78%	99.20%	96.50%	99.13%
0.20	79.28%	98.48%	76.09%	98.18%
0.30	65.14%	96.87%	56.44%	95.63%

Table 2: Accuracy under FGSM and PGD attacks for the baseline and adversarially trained models.

5.3 Robustness Gains

To make the effect of adversarial training explicit, Table 3 reports the improvement in accuracy over the baseline in percentage points.

ε	FGSM gain	PGD gain
0.00	+0.03	+0.03
0.01	+0.12	+0.12
0.05	+0.65	+0.69
0.10	+2.41	+2.63
0.20	+19.20	+22.10
0.30	+31.73	+39.19

Table 3: Accuracy gains from adversarial training, measured in percentage points.

These results show three clear patterns. First, the baseline model is highly accurate on clean data but increasingly fragile under adversarial perturbations. Its accuracy falls sharply as ε increases, especially at $\varepsilon = 0.20$ and $\varepsilon = 0.30$. Second, PGD is consistently stronger than FGSM at the same perturbation strength, which matches expectations because PGD performs iterative optimization rather than a single update step. Third, adversarial training substantially improves robustness across the entire epsilon sweep while preserving essentially the same clean accuracy.

The most important result occurs at $\varepsilon = 0.30$. Under FGSM, the baseline model drops to 65.14% accuracy, while the adversarially trained model retains 96.87%. Under PGD, the baseline drops further to 56.44%, while the adversarially trained model still achieves 95.63%. This demonstrates that adversarial training is highly effective in this IDS setting.

6 Challenges and Limitations

The project has several limitations that affect how broadly the results can be generalized. First, perturbations were restricted to continuous features only. This preserves semantic validity for tabular data, but it may understate worst-case vulnerability because categorical feature manipulations were not considered. Second, the experiments were conducted on a single dataset, NSL-KDD, so the results may not transfer directly to other IDS benchmarks or modern network telemetry datasets. Third, we evaluated only one model family, an MLP, which limits conclusions about architecture-specific robustness. Fourth, adversarial training used a fixed training perturbation of $\varepsilon = 0.1$, which may not be optimal for all attack strengths. Finally, PGD evaluation and adversarial training both increase runtime cost, which matters for any real-time deployment scenario.

7 Conclusion

This project evaluated the adversarial robustness of a neural network-based intrusion detection system under FGSM and PGD white-box attacks. The baseline MLP achieved excellent clean-data accuracy, but it proved highly vulnerable to adversarial perturbations, especially under PGD. Adversarial training dramatically improved robustness across the full epsilon sweep while preserving nearly identical clean accuracy. Overall, the results show that nominally accurate IDS models can still be fragile under adversarial pressure, but adversarial training provides a practical and effective defense in this setting.

8 Future Work

Several extensions could improve the project and strengthen the conclusions. First, the same evaluation should be repeated on additional intrusion detection datasets to test whether the observed robustness gains generalize beyond NSL-KDD. Second, future work should compare multiple model architectures, including CNNs, transformers, and newer tabular deep learning models, to determine whether robustness trends depend on the underlying model family. Third, adversarial training could be expanded by tuning the training epsilon and by incorporating stronger iterative attacks such as PGD during training rather than FGSM alone. Fourth, additional defenses such as feature denoising, ensemble methods, or certified robustness approaches could be evaluated. Finally, future work should measure computational overhead, inference latency, and deployment trade-offs more directly to better assess practicality in real-world IDS pipelines.

References

[1] I. J. Goodfellow, J. Shlens, and C. Szegedy, *Explaining and Harnessing Adversarial Examples*, International Conference on Learning Representations, 2015.

[2] A. E. Cina et al., *Wild Patterns Reloaded: A Survey of Machine Learning Security against Training Data Poisoning*, 2023.

[3] V. G. Costa et al., *How Deep Learning Sees the World: A Survey on Adversarial Attacks & Defenses*, 2024.

[4] H. Li et al., *A Review of Adversarial Attack and Defense for Classification Methods*, 2022.

[5] Y. Li et al., *Adversarial Defense in Modulation Recognition via Diffusion and Segment-Wise Classification*, 2024.

[6] Canadian Institute for Cybersecurity, University of New Brunswick, *NSL-KDD Dataset*. https://www.unb.ca/cic/datasets/nsl.html